



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ	Информатика и системы управления (ИУ)
КАФЕДРА	Информационная безопасность (ИУ8)
ДИСЦИПЛИНА	Математические основы искусственного интеллекта

ДОМАШНЕЕ ЗАДАНИЕ №3

Методы многомерного поиска

название

Группа ИУ8-13М

Студент	5 вариант		Никулина З.Е.
		<i>подпись</i>	<i>фамилия, и.о.</i>

Преподаватель		Каганов Ю.Т.
	<i>подпись</i>	<i>фамилия, и.о.</i>

Москва, 2025 г.

Цель работы – изучение алгоритмов многомерного поиска 1-го и 2-го порядка, разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка, вычисление экстремумов функции.

Практическая часть

1 Постановка задачи

Целью работы является исследование сходимости различных численных методов оптимизации при нахождении минимума тестовой функции Розенброка, задаваемой выражением:

$$f(x) = \sum_{i=1}^{n-1} [a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2] + f_0, \quad (1)$$

где $a = 70$, $b = 5$, $f_0 = 30$, $n = 3$.

В ходе выполнения задачи требуется определить стационарные точки и значения функции, реализовать методы Флетчера–Ривза, Полака–Рибьера, Давидона–Флетчера–Пауэлла и Левенберга–Марквардта, оценить точность и скорость их сходимости при одинаковых начальных условиях, а также провести сравнение полученных результатов с визуализацией поверхности функции и траекторий оптимизации.

2 Исследование минимума

В качестве начального приближения выбрана точка $x_0(0, 0, 0)$, что обеспечивает движение алгоритмов из области, удалённой от минимума и позволяет оценить их устойчивость и скорость сходимости.

В результате анализа стационарных точек получена единственная точка $x^*(1, 1, 1)$, соответствующая глобальному минимуму функции $f(x^*) = 30$. Все методы при заданной точности $\varepsilon = 10^{-6}$ сходятся к этой точке, что подтверждает корректность их реализации.

При старте из $x_0(0, 0, 0)$ методы демонстрируют различную скорость и характер сходимости. На рисунке 1 показаны характеристики методов при

поиске минимума. На рисунке 2 и 3 изображены графические результаты для оценки сходимости и времени, затраченного моделями, соответственно.

CG Fletcher-Reeves	iters= 107	time=0.0107s	f*=30	x*=[1.00000005 1.00000009 1.00000018]	grad =9.42e-07
CG Polak-Ribière	iters= 275	time=0.0122s	f*=30	x*=[1.00000002 1.00000005 1.00000001]	grad =6.25e-07
DFP Quasi-Newton	iters= 42	time=0.0015s	f*=30	x*=[1. 1. 0.99999999]	grad =5.17e-07
Levenberg-Marquardt	iters= 17	time=0.0000s	f*=30	x*=[1. 0.99999999 0.99999999]	grad =3.00e-08

Рисунок 1 – Сравнение методов для $x_0(0, 0, 0)$

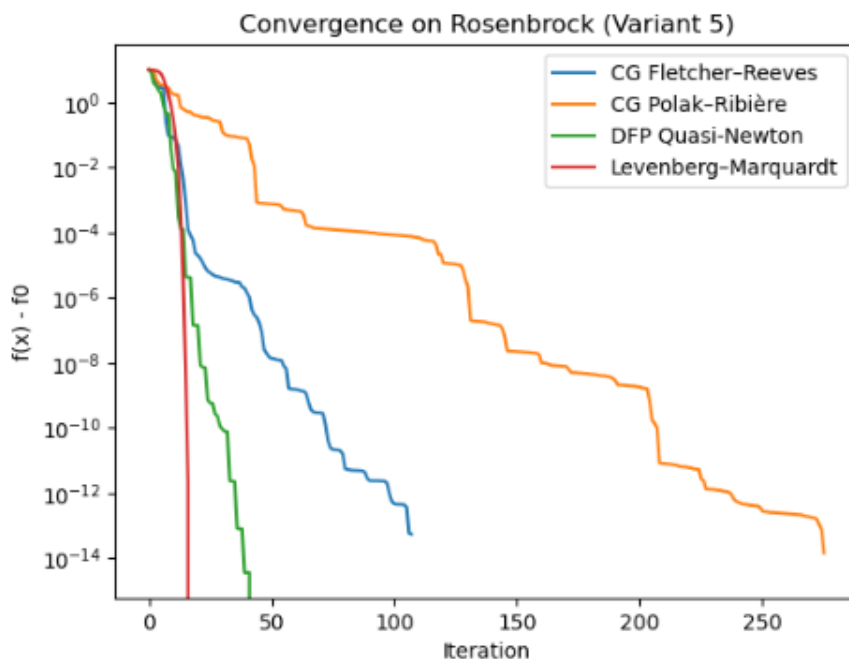


Рисунок 2 – Сходимость методов для $x_0(0, 0, 0)$

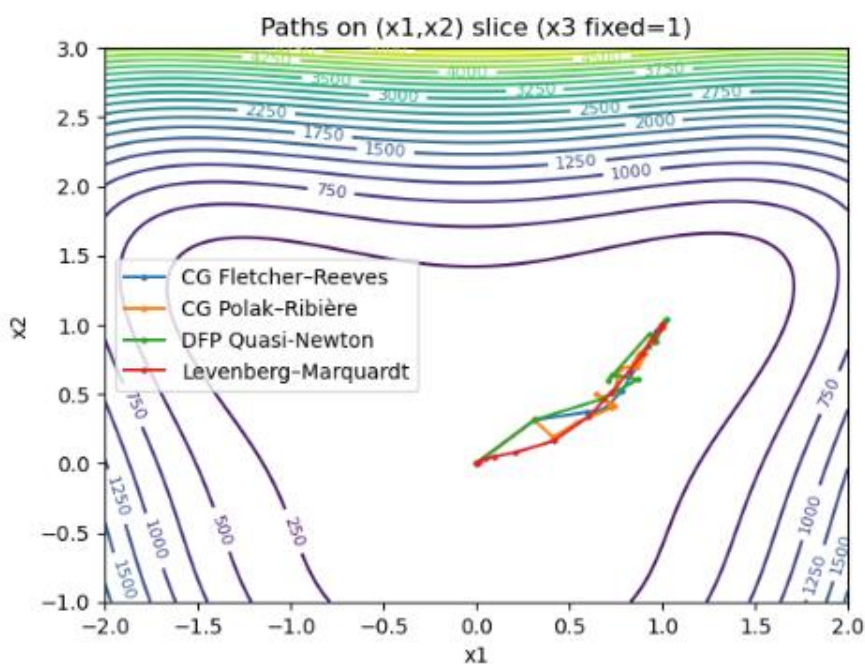


Рисунок 3 – Работа методов для $x_0(0, 0, 0)$

Согласно рисунку 4 Метод Левенберга–Марквардта показал наилучшую скорость: 17 итераций и наименьшее значение нормы градиента, что связано с его демпфированием и корректировкой шага вблизи минимума. Метод Девидона–Флетчера–Пауэлла также продемонстрировал высокую эффективность (42 итерации), обеспечив быструю стабилизацию и точность. Методы сопряжённых градиентов оказались более чувствительными к искривлению Розенброка: Флетчер–Ривз сошёлся за 107 итераций, а Полак–Рибьер — за 275, при этом их траектории имели колебательный характер и более медленно приближались к минимуму.

```
#1: x* = (1.0000000000, 1.0000000000, 1.0000000000), f(x*) = 30
```

Сравнение методов

Метод	Итерации	Время, с	grad	f*	x*
Levenberg-Marquardt	17	0.005100	3.00e-08	30	[1.00000000 0.99999999 0.99999999]
DFP Quasi-Newton	42	0.000000	5.17e-07	30	[1.00000000 1.00000000 0.99999999]
CG Fletcher-Reeves	107	0.012759	9.42e-07	30	[1.00000005 1.00000009 1.00000018]
CG Polak-Ribière	275	0.012915	6.25e-07	30	[1.00000002 1.00000005 1.00000010]

Рисунок 4 – Оценка скорости сходимости и времени для $x_0(0, 0, 0)$

При выборе более удаленной точки от минимума были получены метрики, представленные на рисунке 5.

```
CG Fletcher-Reeves | iters= 410 time=0.0234s f*=30 x*=[1.00000002 1.00000003 1.00000006] ||grad||=1.34e-06
CG Polak-Ribière | iters= 123 time=0.0120s f*=30 x*=[1.00000027 1.00000053 1.00000107] ||grad||=8.73e-06
DFP Quasi-Newton | iters= 45 time=0.0020s f*=30 x*=[1. 1. 1.00000001] ||grad||=4.98e-07
Levenberg-Marquardt | iters= 18 time=0.0000s f*=30 x*=[1. 1. 1.] ||grad||=2.78e-09
```

Рисунок 5 – Сравнение методов для $x_0(-1.5, 1.2, 0.8)$

График зависимости на рисунке 6 показывает, что Девидона–Флетчера–Пауэлла и Левенберг–Марквардт сходятся экспоненциально быстро, тогда как методы сопряжённых градиентов демонстрируют ступенчатый характер убывания функции. На рисунке 7 видно, что траектории Девидона–Флетчера–Пауэлла и Левенберг–Марквардт направлены почти прямо к минимуму, тогда как траектории методов сопряженных градиентов совершают многочисленные колебания вдоль долины.

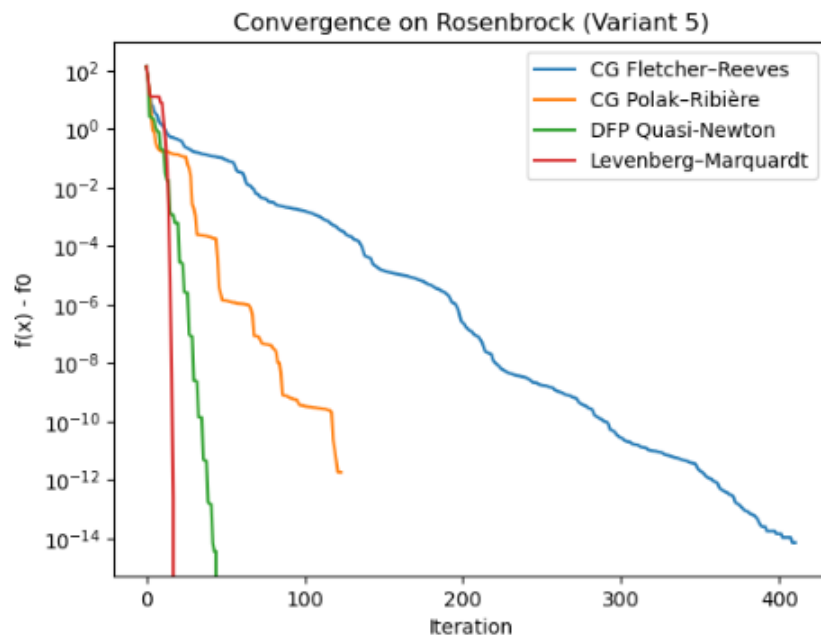


Рисунок 6 – Сходимость методов для $x_0(-1.5, 1.2, 0.8)$

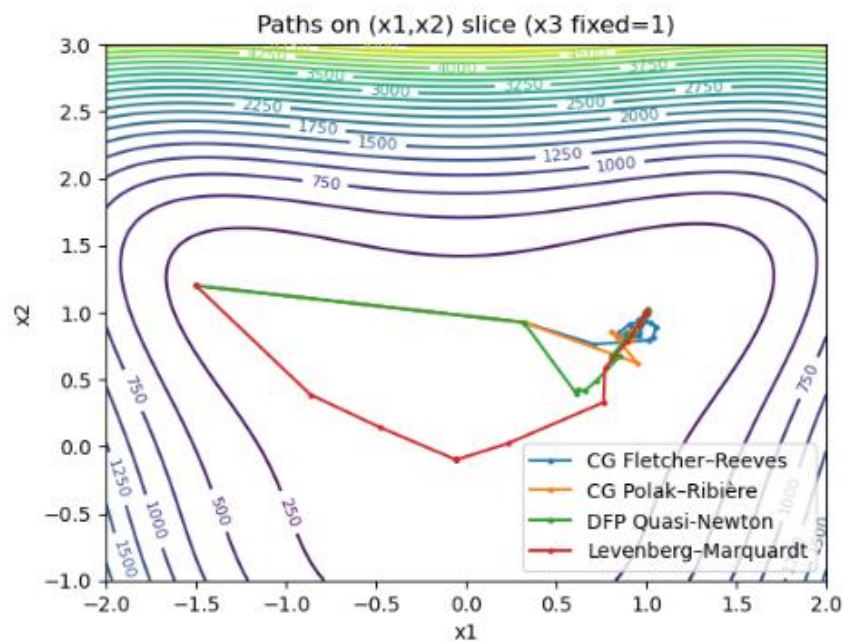


Рисунок 7 – Работа методов для $x_0(-1.5, 1.2, 0.8)$

По результатам на рисунке 8: метод Левенберга–Марквардта показал наиболее быструю сходимость — всего 18 итераций до достижения минимума, однако при этом при слишком большом параметре демпфирования на первых шагах наблюдалось «перескакивание» через долину и увеличение значения функции до $f = 37$, после чего алгоритм стабилизировался. Метод Давидона–Флетчера–Пауэлла обеспечил высокую точность и стабильную сходимость — за 45 итераций. Методы сопряжённых градиентов показали более медленную

сходимость. Алгоритм Флетчера–Ривза потребовал 410 итераций, а метод Полака–Рибьера — 123. Это объясняется тем, что при движении вдоль узкой долины направление градиента часто перестаёт быть оптимальным, и методам приходится корректировать направление движения на каждом шаге.

#1: $x^* = (1.0000000000, 1.0000000000, 1.0000000000)$, $f(x^*) = 30$

Сравнение методов

Метод	Итерации	Время, с	$\ grad\ $	f^*	x^*
Levenberg-Marquardt	18	0.000000	2.78e-09	30	[1.00000000 1.00000000 1.00000000]
DFP Quasi-Newton	45	0.002000	4.98e-07	30	[1.00000000 1.00000000 1.00000001]
CG Polak-Ribière	123	0.011997	8.73e-06	30	[1.00000027 1.00000053 1.00000107]
CG Fletcher-Reeves	410	0.023439	1.34e-06	30	[1.00000002 1.00000003 1.00000006]

Рисунок 8 – Оценка скорости сходимости и времени для $x_0(-1.5, 1.2, 0.8)$

Изображение функции при фиксированном значении $x_3 = 1$ показан на рисунке 9.

3D-график функции Розенброка

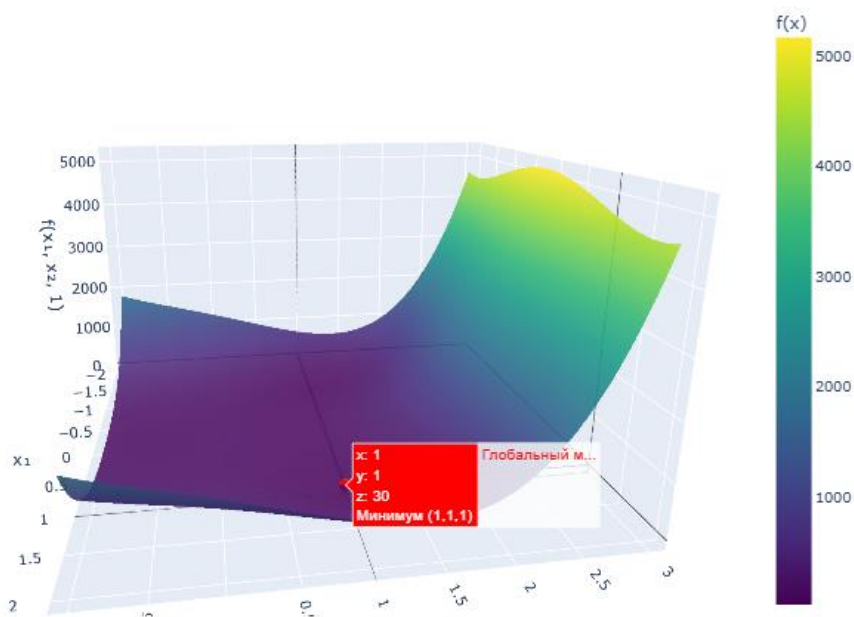


Рисунок 9 – Графическое изображение функции 3d

Таким образом, по сочетанию точности, скорости и устойчивости оптимальным для данной задачи является метод Давидона–Флетчера–Пауэлла, а метод Левенберга–Марквардта показал лучшую скорость при правильно выбранных параметрах.

Листинги программ приведены в Приложении А.

ПРИЛОЖЕНИЕ А

Листинги программ

```
import numpy as np, time, math
import matplotlib.pyplot as plt
import pandas as pd
import sympy as sp
from pathlib import Path
import plotly.graph_objects as go
a, b, f0, n = 70.0, 5.0, 30.0, 3

def f(x, a=a, b=b, f0=f0):
    x = np.asarray(x, float)
    s = 0.0
    for i in range(n-1):
        s += a*(x[i]**2 - x[i+1])**2 + b*(x[i]-1.0)**2
    return s + f0

def gradf(x, a=a, b=b):
    x = np.asarray(x, float)
    g = np.zeros_like(x)
    for i in range(n-1):
        g[i] += 4*a*(x[i]**2 - x[i+1])*x[i] + 2*b*(x[i]-1.0)
        g[i+1] += -2*a*(x[i]**2 - x[i+1])
    return g

def hessf(x, a=a, b=b):
    x = np.asarray(x, float)
    H = np.zeros((n,n))
    for i in range(n-1):
        H[i,i] += 12*a*x[i]**2 - 4*a*x[i+1] + 2*b
        H[i,i+1] += -4*a*x[i]
        H[i+1,i] += -4*a*x[i]
        H[i+1,i+1] += 2*a
    return H

def _n(v): return float(np.linalg.norm(v))

def strong_wolfe_line_search(F, G, xk, pk, c1=1e-4, c2=0.9, alpha0=1.0, max_iter=20):
    alpha, fk, gk = alpha0, F(xk), G(xk)
    gkpk = float(gk @ pk)
    if gkpk >= 0:
        pk = -gk
        gkpk = float(gk @ pk)
    for _ in range(max_iter):
        x1 = xk + alpha*pk
        f1 = F(x1)
        if f1 <= fk + c1*alpha*gkpk:
            g1 = G(x1)
            if abs(float(g1 @ pk)) <= -c2*gkpk:
                return alpha, x1, f1, g1
            alpha *= 0.5
        x1 = xk + alpha*pk
    return alpha, x1, F(x1), G(x1)

def converged(x_hist, f_hist, g_hist, eps_g=1e-6, eps_x=1e-9, eps_f=1e-12):
    if _n(g_hist[-1]) < eps_g: return True, "||grad|| threshold"
    if len(x_hist) >= 2:
        if _n(x_hist[-1]-x_hist[-2]) < eps_x and abs(f_hist[-1]-f_hist[-2]) < eps_f:
            return True, "Δx & Δf thresholds"
```

```

    return False, ""

def cg_fletcher_reeves(x0, max_iter=10_000):
    xk = np.asarray(x0, float)
    gk = gradf(xk)
    pk = -gk
    x_hist, f_hist, g_hist = [xk.copy()], [f(xk)], [gk.copy()]
    t0 = time.time()
    for _ in range(max_iter):
        a_, x1, f1, g1 = strong_search(f, gradf, xk, pk)
        beta = float(g1 @ g1) / max(1e-32, float(g_hist[-1] @ g_hist[-1]))
        pk = -g1 + beta*pk
        xk, gk = x1, g1
        x_hist.append(xk.copy()); f_hist.append(f1); g_hist.append(gk.copy())
        ok, _ = converged(x_hist, f_hist, g_hist)
        if ok: break
    return xk, f_hist[-1], dict(x_hist=np.array(x_hist), f_hist=np.array(f_hist),
                               g_hist=np.array(g_hist), iters=len(f_hist)-1,
                               time=time.time()-t0)

def cg_polak_ribiere(x0, max_iter=10_000):
    xk = np.asarray(x0, float)
    gk = gradf(xk)
    pk = -gk
    x_hist, f_hist, g_hist = [xk.copy()], [f(xk)], [gk.copy()]
    t0 = time.time()
    for k in range(max_iter):
        a_, x1, f1, g1 = strong_search(f, gradf, xk, pk)
        yk = g1 - gk
        beta = max(0.0, float(g1 @ yk) / max(1e-32, float(gk @ gk)))
        pk = -g1 + beta*pk
        xk, gk = x1, g1
        x_hist.append(xk.copy()); f_hist.append(f1); g_hist.append(gk.copy())
        if (k+1) % n == 0:
            pk = -gk
        ok, _ = converged(x_hist, f_hist, g_hist)
        if ok: break
    return xk, f_hist[-1], dict(x_hist=np.array(x_hist), f_hist=np.array(f_hist),
                               g_hist=np.array(g_hist), iters=len(f_hist)-1,
                               time=time.time()-t0)

def dfp(x0, max_iter=10_000):
    xk = np.asarray(x0, float)
    H = np.eye(n)
    gk = gradf(xk)

    x_hist, f_hist, g_hist = [xk.copy()], [f(xk)], [gk.copy()]
    t0 = time.time()

    for k in range(max_iter):
        pk = -H @ gk
        _, x1, f1, g1 = strong_search(f, gradf, xk, pk)

        s = (x1 - xk).astype(float)
        y = (g1 - gk).astype(float)

        sty = float(s @ y)
        if sty <= 1e-20 or not np.isfinite(sty):
            H = np.eye(n)
        else:
            Hy = H @ y
            yHy = float(y @ Hy)

```

```

        # DFP:  $H_{k+1} = H_k + (s s^T)/(s^T y) - (H y y^T H)/(y^T H y)$ 
        H = H + np.outer(s, s)/sty - np.outer(Hy, Hy)/max(1e-32, yHy)

    xk, gk = x1, g1
    x_hist.append(xk.copy()); f_hist.append(f1); g_hist.append(gk.copy())

    if (k+1) % n == 0 and k > 0:
        H = np.eye(n)

    ok, _ = converged(x_hist, f_hist, g_hist)
    if ok: break

    return xk, f_hist[-1], dict(
        x_hist=np.array(x_hist),
        f_hist=np.array(f_hist),
        g_hist=np.array(g_hist),
        iters=len(f_hist)-1,
        time=time.time()-t0
    )

def residual(x, a=70.0, b=5.0):
    x = np.asarray(x, float)
    #  $r = [\sqrt{a}*(x_1^2-x_2), \sqrt{b}*(x_1-1), \sqrt{a}*(x_2^2-x_3), \sqrt{b}*(x_2-1)]$ 
    r = []
    for i in range(n-1):
        r.append(np.sqrt(a)*(x[i]**2 - x[i+1]))
        r.append(np.sqrt(b)*(x[i] - 1.0))
    return np.array(r)

def jacobian(x, a=70.0, b=5.0):
    x = np.asarray(x, float)
    m = 2*(n-1)
    J = np.zeros((m, n))
    row = 0
    for i in range(n-1):
        #  $r_{2i} = \sqrt{a}*(x_i^2 - x_{i+1})$ 
        J[row, i] = 2*np.sqrt(a)*x[i]
        J[row, i+1] = -np.sqrt(a)
        row += 1
        #  $r_{2i+1} = \sqrt{b}*(x_i - 1)$ 
        J[row, i] = np.sqrt(b)
        row += 1
    return J

def levenberg_marquardt(x0, mu0=1e-2, nu0=2.0, max_iter=200, tol_g=1e-8):
    xk = np.asarray(x0, float)
    mu, nu = float(mu0), float(nu0)
    x_hist, f_hist, g_hist = [xk.copy()], [f(xk)], [gradf(xk)]
    for _ in range(max_iter):
        r = residual(xk)
        J = jacobian(xk)
        g = J.T @ r # это градиент  $1/2 ||r||^2$  (без f0)
        A = J.T @ J # приближённый гессиан (Гаусса-Ньютона)
        try:
            d = np.linalg.solve(A + mu*np.eye(n), -g)
        except np.linalg.LinAlgError:
            d = -g

        # Предсказанное убывание (квадратичная модель)
        pred = - (g @ d + 0.5 * d @ (A @ d))
        f_curr = f(xk)
        x_new = xk + d

```

```

f_new = f(x_new)
rho = (f_curr - f_new) / max(1e-32, pred)

if rho > 0:          # шаг удачный -> принимаем и уменьшаем демпфирование
    xk = x_new
    mu = mu * max(1/3, 1 - (2*rho - 1)**3)
    nu = 2.0
else:               # шаг плохой -> усиливаем демпфирование
    mu = mu * nu
    nu = 2.0 * nu

x_hist.append(xk.copy()); f_hist.append(f(xk)); g_hist.append(gradf(xk))
t0 = time.time()
if np.linalg.norm(g_hist[-1]) < tol_g:
    break

return xk, f_hist[-1], dict(x_hist=np.array(x_hist),
                           f_hist=np.array(f_hist),
                           g_hist=np.array(g_hist),
                           iters=len(f_hist)-1,
                           time=time.time()-t0)

x0 = np.array([0, 0, 0])

methods = {
    "CG Fletcher-Reeves": cg_fletcher_reeves,
    "CG Polak-Ribière":   cg_polak_ribiere,
    "DFP Quasi-Newton":   dfp,
    "Levenberg-Marquardt": levenberg_marquardt,
}

results = {}
for name, algo in methods.items():
    xs, fs, info = algo(x0)
    results[name] = (xs, fs, info)
    print(f"{name:>24s} | iters={info['iters']:4d} time={info['time']:.4f}s "
          f"f*={fs:.6g} x*={xs} ||grad|={_n(info['g_hist'][-1]):.2e}")

plt.figure()
for name, (_, _, info) in results.items():
    plt.semilogy(info["f_hist"] - f0, label=name)
plt.xlabel("Iteration")
plt.ylabel("f(x) - f0")
plt.title("Convergence on Rosenbrock (Variant 5)")
plt.legend(); plt.show()

x1 = np.linspace(-2.0, 2.0, 300)
x2 = np.linspace(-1.0, 3.0, 300)
X1, X2 = np.meshgrid(x1, x2)
X3 = 1.0
Z = a*(X1**2 - X2)**2 + b*(X1-1)**2 + a*((X2**2 - X3)**2) + b*(X2-1)**2 + f0
plt.figure()
cs = plt.contour(X1, X2, Z, levels=20); plt.clabel(cs, inline=1, fontsize=8)
for name, (_, _, info) in results.items():
    path = info["x_hist"]
    plt.plot(path[:,0], path[:,1], marker='o', markersize=2, label=name)
plt.xlabel("x1"); plt.ylabel("x2"); plt.title("Paths on (x1,x2) slice (x3 fixed=1)")
plt.legend(); plt.show()

def f_ros(x, a=a, b=b, f0=f0):
    x = np.asarray(x, float)
    s = 0.0
    for i in range(n-1):

```

```

        s += a*(x[i]**2 - x[i+1])**2 + b*(x[i]-1.0)**2
    return s + f0

x1, x2, x3 = sp.symbols('x1 x2 x3', real=True)
a_s, b_s = sp.simplify(a), sp.simplify(b)
g1 = 4*a_s*(x1**2 - x2)*x1 + 2*b_s*(x1 - 1)
g2 = -2*a_s*(x1**2 - x2) + 4*a_s*(x2**2 - x3)*x2 + 2*b_s*(x2 - 1)
g3 = -2*a_s*(x2**2 - x3)

sols = sp.solve([sp.Eq(g1,0), sp.Eq(g2,0), sp.Eq(g3,0)], [x1, x2, x3],
domain=sp.S.Complexes)
sols = [tuple(sp.N(s, 12) for s in sol) for sol in sols]

for i, (sx1, sx2, sx3) in enumerate(sols, 1):
    xv = np.array([float(sx1), float(sx2), float(sx3)], float)
    print(f"#{i}: x* = ({sx1}, {sx2}, {sx3}), f(x*) = {f_ros(xv):.12g}")
print("")

if 'results' not in globals():
    raise RuntimeError("")

rows = []
for name, (x_star, f_star, info) in results.items():
    rows.append({
        "Метод": name,
        "Итерации": int(info["iters"]),
        "Время, с": float(info["time"]),
        "||grad||": float(np.linalg.norm(info["g_hist"][-1])),
        "f*": float(f_star),
        "x*": "[" + " ".join(f"{v:.8f}" for v in np.asarray(x_star, float)) + "]",
    })

df = (pd.DataFrame(rows)
      .sort_values(["Итерации", "Время, с"], ascending=[True, True])
      .reset_index(drop=True))

print("\nСравнение методов")
print(df.to_string(index=False, formatters={
    "Время, с": lambda x: f"{x:.6f}",
    "||grad||": lambda x: f"{x:.2e}",
    "f*": lambda x: f"{x:.12g}",
}))

a, b, f0 = 70.0, 5.0, 30.0

x1 = np.linspace(-2.0, 2.0, 200)
x2 = np.linspace(-1.0, 3.0, 200)
X1, X2 = np.meshgrid(x1, x2)
X3 = 1.0

Z = a*(X1**2 - X2)**2 + b*(X1 - 1)**2 + a*((X2**2 - X3)**2) + b*(X2 - 1)**2 + f0

surf = go.Surface(
    x=X1, y=X2, z=Z,
    colorscale="Viridis",
    opacity=0.9,
    colorbar=dict(title="f(x)")
)

min_trace = go.Scatter3d(
    x=[1], y=[1], z=[30],
    mode='markers+text',

```

```

        marker=dict(size=6, color='red'),
        text=["Минимум (1,1,1)"],
        textposition="top right",
        name="Глобальный минимум"
    )
fig = go.Figure(data=[surf, min_trace])
fig.update_layout(
    title="3D-график функции Розенброка",
    scene=dict(
        xaxis_title='x1',
        yaxis_title='x2',
        zaxis_title='f(x1, x2, 1)',
        aspectratio=dict(x=1, y=1, z=0.6)
    ),
    width=900,
    height=700
)
fig.show()

```